

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет прикладної математики та інформатики
Кафедра програмування

КУРСОВА РОБОТА

"Розробка клієнт-серверного застосунку обміну повідомленнями"

Виконала:
студентка III курсу, групи ПМІ-36
напряму підготовки (спеціальності)
122 - "Комп'ютерні науки"

Тригуб Марта
Науковий керівник:
Ас. Жировецький В.В.

ЗМІСТ

Вступ.....	4
1. Аналіз предметної області та постановка задачі.....	5
1.1. Існуючі рішення у сфері обміну повідомленнями.....	5
1.2. Вимоги до системи обміну повідомленнями.....	6
1.2.1. Функціональні вимоги.....	6
1.2.2. Нефункціональні вимоги.....	7
1.3. Постановка задачі розробки.....	7
2. Опис системи.....	9
2.1. Архітектура проекту.....	9
2.1.1. Клієнт-серверна архітектура з MVC-шаблоном.....	9
2.1.2. Переваги і недоліки клієнт-серверної архітектури з MVC-шаблоном.....	10
2.2. Використані технології та бібліотеки.....	12
2.2.1. WebSocket.....	12
2.2.2. Django.....	14
2.2.3. Django Channels.....	15
2.2.4. ASGI-сервер (Daphne).....	16
2.2.5. PostgreSQL.....	17
2.2.6. HTMX.....	18
2.2.7. Pillow.....	19
2.2.8. JavaScript WebSocket API.....	19
3. Огляд застосунку.....	19
3.1. Ролі користувачів.....	19
3.2. Бізнес логіка застосунку.....	20
3.2.1. Процес реєстрації та автентифікації.....	20

	3
3.2.2. Обмін повідомленнями у реальному часі.....	22
3.2.3. Відображення статусу користувачів.....	23
3.2.4. Історія повідомлень.....	24
3.3. Безпека та обмеження.....	24
3.3.1. Авторизація та доступ користувачів.....	24
3.3.2. Рольові обмеження.....	25
3.3.3. Захист від загроз безпеки.....	26
3.3.4. Обмеження на розмір та тип контенту.....	26
Висновки.....	27
Список використаних джерел.....	28
Додаток А.....	29

Вступ

У сучасному світі, який динамічно розвивається ефективна комунікація стає важливою частиною повсякденного життя. Потреба у швидкому обміні інформацією відчувається як у корпоративних середовищах - навчальних закладах, офісах, виробничих приміщеннях, так і в особистому спілкуванні між людьми. На ринку представлено багато високоякісних комунікаційних платформ, проте більшість з них орієнтовані на глобальне використання та зберігання даних на зовнішніх серверах, що не завжди відповідає вимогам організацій щодо конфіденційності та контролю над даними.

Метою цієї роботи є розробка клієнт-серверного застосунку обміну повідомленнями, який забезпечить швидку та захищену комунікацію між користувачами в межах однієї організації без залежності від зовнішніх сервісів.

Для того, щоб досягти цієї мети необхідно виконати певні завдання, а саме:

- Проаналізувати сучасні технології обміну повідомленнями в реальному часі
- Обрати архітектуру та спроектувати систему
- Розробити серверну частину застосунку на основі WebSockets
- Створити зручний та інтуїтивно зрозумілий клієнтський інтерфейс
- Реалізувати базовий функціонал обміну повідомленнями та групового чату
- Забезпечити відображення статусу користувачів та історії повідомлень
- Протестувати застосунок

1. Аналіз предметної області та постановка задачі

1.1. Існуючі рішення у сфері обміну повідомленнями

Сучасний ринок пропонує широкий вибір застосунків для обміну повідомленнями, які використовуються як для професійних потреб, так і для особистого спілкування. Аналіз вже існуючих рішень дозволяє визначити їхні переваги та недоліки.

Серед глобальних месенджерів можна виділити:

- **WhatsApp** – один з найпоширеніших месенджерів у світі з великою кількістю активних користувачів. Підтримує голосові та відеодзвінки, обмін файлами. Однак, його використання вимагає зберігання даних на зовнішніх серверах, що не завжди прийнятно для корпоративного використання.
- **Microsoft Teams** – платформа для корпоративної комунікації та командної роботи. Підтримує відеоконференції, інтегрується з іншими продуктами від Microsoft. Однак, для невеликих організацій є складність у налаштуванні, а також висока вартість.

Серед рішень, орієнтованих на корпоративне використання та внутрішні мережі організацій, можна виділити:

- **Mattermost** – платформа обміну повідомленнями для командної роботи. Інтегрується з іншими сервісами, але має недолік у складності налаштування та адміністрування для непідготовлених користувачів.
- **Zulip** – месенджер з унікальною системою потоків повідомлень, яка спрощує організацію спілкування, інтегрується з іншими інструментами. Однак, інтерфейс цього месенджера є досить специфічним і потребує звикання користувача.



Рис. 1 Mattermost та Zulip

Більшість популярних месенджерів зберігають дані на зовнішніх серверах, що не завжди підходить під вимоги безпеки для корпоративного використання. Також ці рішення часто вимагають технічних знань для встановлення та налаштування, містять функції, які непотрібні для простого обміну повідомленнями і мають значну вартість ліцензій.

Проаналізувавши недоліки існуючих рішень для використання в корпоративному середовищі, можна дійти до висновку, що розробка власного застосунку для обміну повідомленнями є актуальною задачею, яка дозволить створити рішення, оптимізоване під конкретні потреби організацій з повним контролем над даними та інфраструктурою.

1.2. Вимоги до системи обміну повідомленнями

На основі аналізу існуючих рішень та враховуючи специфіку роботи в корпоративному середовищі, можна сформулювати вимоги до системи обміну повідомленнями. Їх можна розділити на дві групи: функціональні та нефункціональні.

1.2.1. Функціональні вимоги

- Реєстрація та авторизація користувачів: Обліковий запис повинен створюватись з унікальним логіном, захищена автентифікація користувачів, а також сесія користувача має зберігатися.

- Обмін текстовими повідомленнями: Надсилання та отримання повідомлень має відбуватись у реальному часі, повинна бути індикація набору тексту повідомлення та прочитання повідомлень.
- Історія повідомлень: Збереження історії групових повідомлень, відображення статусу активності користувачів (онлайн/офлайн).

1.2.2. Нефункціональні вимоги

- Продуктивність: Доставка повідомлень має бути миттєва (затримка не більше 1 секунди), підтримка одночасної роботи множини користувачів організації, оптимальне використання ресурсів сервера.
- Надійність: Система повинна працювати стабільно, дані мають зберігатись при аварійному завершенні роботи, з'єднання повинно автоматично відновлятись при його втраті.
- Безпека: Шифрування повідомлень та захист від несанкціонованого доступу.
- Масштабованість: Можливість розширення функціональності і підтримка зростання бази користувачів без значної втрати продуктивності.
- Зручність використання: Інтуїтивно зрозумілий інтерфейс.

1.3. Постановка задачі розробки

Виходячи з проведеного аналізу предметної області та сформованих вимог були визначені ключові завдання, які охоплюють концептуальні, технічні та організаційні аспекти розробки клієнт-серверного застосунку обміну повідомленнями для корпоративних середовищ. Насамперед, необхідно розробити архітектуру застосунку, яка буде орієнтована на ефективну роботу в межах організації, але водночас передбачатиме можливість масштабування в разі розширення функціональності чи збільшення кількості користувачів. Особливу увагу слід приділити

проектуванню моделі даних, яка буде забезпечувати зручне та надійне зберігання інформації про користувачів і повідомлення.

Однією з основних задач є створення механізму обміну повідомленнями в режимі реального часу, що реалізується за допомогою технології WebSocket. Крім того, важливо забезпечити надійну систему автентифікації та авторизації користувачів, враховуючи специфіку та функціонування в корпоративному середовищі, де велика увага приділяється безпеці та зручності входу в систему.

З технічного боку, реалізація серверної частини здійснюється з використанням фреймворку Django з інтеграцією WebSocket, що дозволяє організувати миттєву передачу повідомлень між клієнтами. Для зберігання інформації використовується база даних PostgreSQL, яка забезпечує швидкий доступ до даних і стабільну роботу застосунку. Клієнтська частина повинна мати зручний та зрозумілий інтерфейс, який підтримуватиме всі необхідні функції для спілкування користувачів.

Організаційно передбачено тестування застосунку в умовах, максимально наближених до реальних сценаріїв використання, що дозволить виявити недоліки ще до впровадження.

Таким чином, основна задача полягає в розробці надійної, безпечної та зручної системи обміну повідомленнями, яка буде працювати в корпоративному середовищі, забезпечуючи комунікацію між користувачами в реальному часі з повним контролем над даними та можливістю централізованого керування.

2. Опис системи

2.1. Архітектура проекту

2.1.1. Клієнт-серверна архітектура з MVC-шаблоном

Клієнт-серверна архітектура з MVC-шаблоном є класичним підходом до розробки додатків, яка поєднує переваги розподіленої обчислювальної архітектури з структурованим підходом до організації коду. Суть цього підходу полягає в побудові системи, яка складається з двох основних частин: клієнтської та серверної, де серверна частина організована відповідно до архітектурного шаблону Model-View-Controller.

Модель (Model) відповідає за управління даними та бізнес-логікою додатку. Вона інкапсулює правила обробки даних, взаємодіє з базою даних, виконує валідацію інформації та реалізує основні алгоритми функціонування системи. Представлення (View) відповідає за відображення даних користувачеві та формування користувацького інтерфейсу. Контролер (Controller) виступає посередником між моделлю та представленням, обробляючи запити користувача, координуючи взаємодію між компонентами та керуючи потоком виконання програми.

У рамках даного проекту використовується модифікація MVC-шаблону, характерна для Django-фреймворку — MVT (Model-View-Template). Це рішення обумовлене специфікою Django, де традиційні ролі MVC-компонентів дещо перерозподілені для оптимізації розробки.

У MVT-архітектурі Django модель (Model) зберігає свою класичну роль управління даними, але реалізується через Django ORM, що забезпечує автоматичне відображення Python-об'єктів у реляційну базу даних. Представлення (View) виконує функції контролера з класичної MVC-архітектури, обробляючи HTTP-запити, викликаючи необхідні

методи моделей та передаючи дані до шаблонів. Шаблон (Template) відповідає за презентаційний рівень, формуючи HTML-відповіді з використанням системи шаблонів Django, що дозволяє вставляти динамічні дані у статичну розмітку.

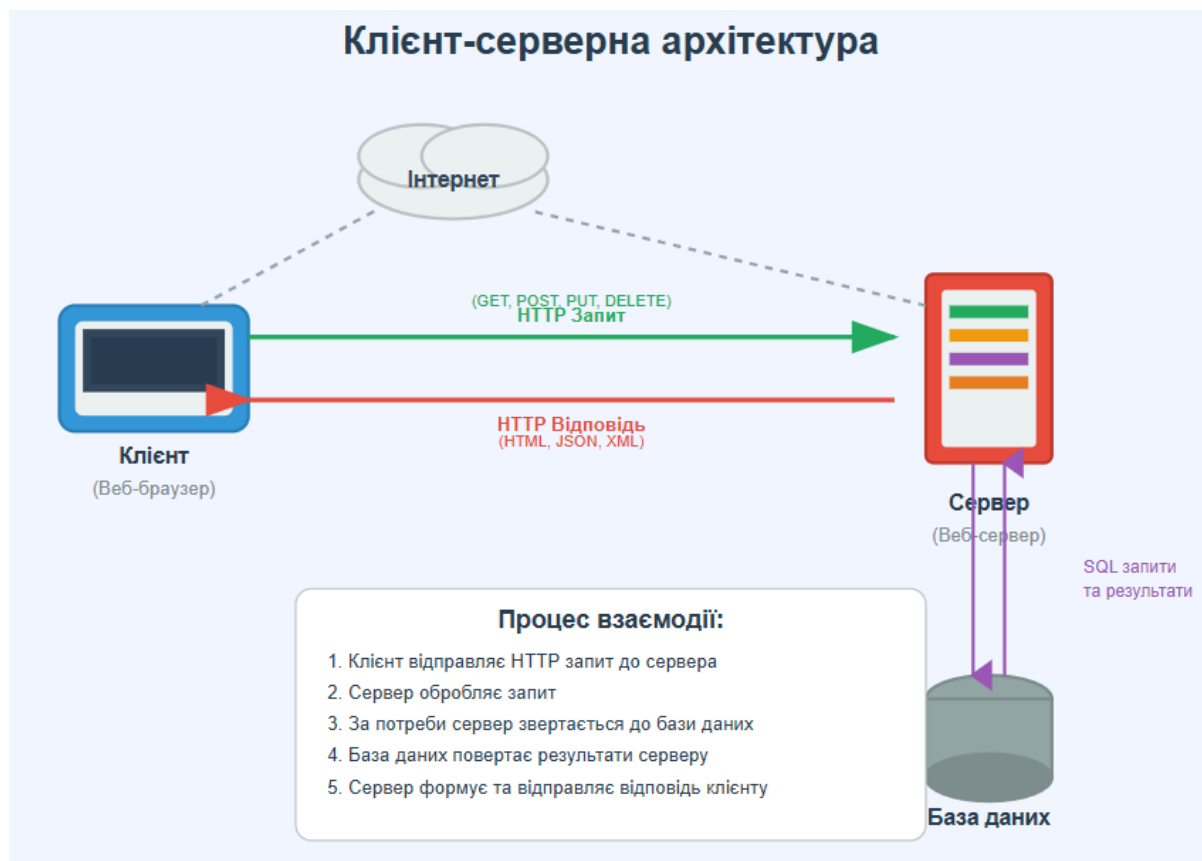


Рис. 2 Клієнт-серверна архітектура

2.1.2. Переваги і недоліки клієнт-серверної архітектури з MVC-шаблоном

Переваги:

- Чіткий поділ: Розділення системи на логічні компоненти (модель, представлення, контролер/шаблон) забезпечує структурованість коду та полегшує розуміння архітектури додатку. Кожен компонент має свою визначену зону відповідальності, що спрощує розробку та подальшу підтримку системи.

- Покращена керованість коду: Модульна структура дозволяє легко орієнтуватися в кодовій базі, швидко знаходити проблеми та вносити зміни в потрібні компоненти без ризику порушення роботи інших частин системи.
- Повторне використання компонентів: Моделі можуть використовуватися різними представленнями та контролерами, представлення можуть працювати з різними моделями, що зменшує дублювання коду та прискорює розробку нового функціоналу.
- Гнучкість розширення: Ця архітектура легко адаптується до зміни вимог, дозволяє додавати нові функції без значної перебудови існуючої системи.

Недоліки:

- Складність системи: Порівняно з простими монолітними рішеннями, архітектура вимагає додаткових зусиль на проектування взаємодії між компонентами, налаштування комунікації та управління розподіленими процесами.
- Мережева залежність: Повна функціональність системи залежить від стабільності мережевого з'єднання між клієнтом та сервером. Проблеми з мережею можуть призводити до недоступності додатку або втрати даних.
- Складність налагодження та тестування: Виявлення та усунення помилок у розподіленій системі потребує спеціалізованих інструментів для моніторингу взаємодії між компонентами. Тестування вимагає емуляції різних сценаріїв мережевої взаємодії.
- Продуктивність: При високому навантаженні сервер може стати критичною точкою відмови, оскільки всі запити обробляються централізовано. Це може призводити до зниження продуктивності всієї системи.

2.2. Використані технології та бібліотеки

2.2.1. WebSocket

WebSocket — це сучасний протокол зв'язку, який дозволяє встановити постійне двостороннє (повнодуплексне) з'єднання між клієнтом та сервером через єдине TCP-з'єднання. Завдяки цьому протоколу обидві сторони можуть обмінюватись даними у реальному часі без потреби у постійному створенні нових запитів. На відміну від класичної моделі HTTP, де передача інформації ініціюється виключно клієнтом, WebSocket дає можливість серверу самостійно надсилати дані у будь-який момент. Тому ця технологія ідеально підходить для реалізації системи обміну повідомленнями в реальному часі.

WebSocket протокол працює у два основні етапи. Спочатку відбувається ініціалізація з'єднання за допомогою стандартного HTTP-запиту, яка називається HTTP-handshake (рукостискання). Після успішного з'єднання клієнт та сервер у WebSocket режим, що дозволяє їм здійснювати обмін даними в обох напрямках. Цей підхід забезпечує сумісність з існуючою інфраструктурою, оскільки початкове з'єднання встановлюється через стандартний HTTP-запит, доповнений спеціальними заголовками Upgrade: websocket та Connection: Upgrade.

Особливості протоколу WebSocket:

- Повнодуплексна комунікація: Клієнт і сервер можуть надсилати дані незалежно один від одного в будь-який момент часу, що забезпечує справжню двосторонню комунікацію без необхідності постійного опитування сервера.
- Низькі накладні витрати: Після встановлення з'єднання WebSocket використовує мінімальні заголовки (2-14 байт) для передачі даних, що суттєво зменшує обсяг трафіку порівняно з HTTP-запитами, заголовки яких можуть сягати сотень байт.

- Постійне з'єднання: Одне TCP-з'єднання залишається активним протягом усієї сесії, що забирає потребу у повторному встановленні з'єднань для кожного запиту і підвищує загальну продуктивність.
- Взаємодія у реальному часі : Миттєва доставка повідомлень без затримок, характерних для підходів polling, long-polling або server-sent events.
- Підтримка різних форматів даних: WebSocket може передавати як текстові дані (UTF-8), так і бінарні дані, що робить його універсальним для різних типів даних.

Значення WebSocket у системі обміну повідомленнями

Використання WebSocket у системі обміну повідомленнями дозволяє реалізувати:

- Миттєву доставку повідомлень всім учасникам чату
- Відображення статусів "користувач друкує" у реальному часі
- Показ онлайн-статусу користувачів та їхньої активності
- Синхронізацію стану додатку між різними вкладками браузера
- Групові чати з множинною доставкою повідомлень
- Передачу мультимедійного контенту без перезавантаження сторінки

```
WebSocket HANDSHAKING /ws/chatroom/public-chat [127.0.0.1:61939]  
WebSocket CONNECT /ws/chatroom/public-chat [127.0.0.1:61939]
```

Рис 3. Процес встановлення з'єднання між клієнтом та сервером
(HTTP-handshake)

2.2.2. Django

Django виступає основним фреймворком для реалізації серверної частини системи обміну повідомленнями. Це високорівневий фреймворк,

написаний мовою Python, який дозволяє швидко створювати безпечні і масштабовані додатки. Підтримка WebSocket реалізується через додаткову бібліотеку Django Channels, що розширює стандартну функціональність Django.

Переваги Django:

- Принцип "batteries included": Django надає великий набір вбудованих інструментів: ORM (система роботи з базами даних), механізми аутентифікації, адміністративна панель, система форм та інші компоненти, що значно пришвидшують розробку типових додатків та дають змогу зосередитися на унікальній бізнес-логіці проекту.
- Високий рівень безпеки: Фреймворк автоматично захищає від поширених вразливостей, таких як CSRF, XSS, SQL-ін'єкції, clickjacking. Надає вбудовані механізми аутентифікації та авторизації, що дозволяє розробляти захищені додатки без зайвих витрат часу.
- Об'єктно-реляційне відображення (ORM): Django ORM дозволяє працювати з базами даних через Python-об'єкти замість написання SQL-запитів вручну. Підтримується автоматичне створення та міграція схем баз даних, а також взаємодія з різними СУБД.
- Масштабованість та продуктивність: Django підтримує кешування, оптимізацію запитів до бази даних, асинхронні операції та може інтегруватися з різними серверами, що робить його придатним для роботи в умовах високого навантаження.
- MVT-архітектура (Model-View-Template): Django реалізує чітке розділення логіки на моделі, представлення та шаблони, що забезпечує високу структурованість коду, полегшує його підтримку та подальший розвиток проекту.

2.2.3. Django Channels

Django Channels – це ключове розширення Django, яке додає підтримку WebSocket та інших асинхронних протоколів. Завдяки Channels дозволяє фреймворк може обробляти не тільки традиційні HTTP-запити, а й довготривалі з'єднання, що є необхідним для створення додатків у реальному часі (real-time applications).

На відміну від стандартного Django, який працює на основі WSGI (Web Server Gateway Interface), Channels використовує ASGI (Asynchronous Server Gateway Interface). Це дозволяє Django працювати в асинхронному середовищі, обробляючи паралельно багато з'єднань, включаючи WebSocket.

Основні компоненти Django Channels:

- **Consumers:** Асинхронні обробники WebSocket-з'єднань, які функціонально подібні до представлень (views) у класичному Django. Consumers обробляють події підключення (connect), отримання повідомлень (receive), надсилання повідомлень (send) та відключення (disconnect) клієнтів. Практичну реалізацію WebSocket Consumer для обробки повідомлень у реальному часі наведено у Додатку А.1.
- **Channel Layers:** Система для передачі повідомлень між різними частинами додатку та між різними серверними процесами. Channel layers забезпечують можливість надсилання повідомлень конкретним користувачам або групам користувачів незалежно від того, до якого серверного процесу вони підключені.

- **Groups:** Засіб логічного об'єднання WebSocket-з'єднань у групи (наприклад, всі учасники певного чату). Це дозволяє ефективно надсилати повідомлення одразу всім учасникам групи.
- **Routing:** Система маршрутизації для WebSocket-з'єднань, подібна до URL routing у Django. Routing визначає, який саме consumer оброблятиме з'єднання залежно від URL-шляху або інших критеріїв. Маршрутизація WebSocket з'єднань показана у Додатку А.2.

2.2.4. ASGI-сервер (Daphne)

Daphne є ASGI-сервером, розробленим командою Django для обробки асинхронних запитів та WebSocket-з'єднань у Django Channels. На відміну від традиційних WSGI-серверів, які обробляють тільки синхронні HTTP-запити, Daphne здатен працювати з довгостроковими з'єднаннями та асинхронними протоколами.

Основні можливості Daphne:

- **Підтримка сучасних протоколів:** Сумісний з HTTP/1.1, HTTP/2 та WebSocket протоколів, що забезпечує гнучкість при створенні сучасних real-time додатків.
- **Асинхронна архітектура:** Завдяки використанню асинхронного циклу подій, Daphne може обробляти велику кількість одночасних з'єднань з мінімальними затримками, що особливо актуально для чатів, сповіщень у реальному часі та інших інтерактивних систем.
- **Інтеграція з Django Channels:** Daphne безпосередньо працює з ASGI-інтерфейсом, що дозволяє йому правильно маршрутизувати запити.
- **Масштабованість:** Підтримка паралельного виконання через кілька робочих процесів дає змогу масштабувати застосунок відповідно до навантаження.

- Управління WebSocket-з'єднаннями: Daphne автоматично керує життєвим циклом WebSocket-з'єднань, включаючи їх встановлення, підтримання та завершення.

2.2.5. PostgreSQL

PostgreSQL використовується як основна система управління базами даних для зберігання всієї інформації системи: користувачів, повідомлень, чатів, груп, налаштувань тощо. PostgreSQL — об'єктно-реляційна СУБД з відкритим вихідним кодом, можливості якої повністю покривають потреби системи обміну повідомленнями. Структура моделей Django для зберігання повідомлень та профілів користувачів у базі даних наведена у Додатку А.3.

Переваги PostgreSQL для даного проекту:

- Велика кількість вбудованих функцій: PostgreSQL надає широкий спектр типів даних, функцій та операторів для роботи з різними видами інформації, включаючи JSON, масиви, часові мітки з часовими поясами.
- Підтримка повнотекстового пошуку: Вбудовані можливості пошуку дозволяють реалізувати ефективний пошук по повідомленнях та користувачах без використання додаткових інструментів індексації.
- ACID-сумісність та надійність: Гарантує цілісність даних навіть при високому навантаженні та збоях системи, що критично важливо для месенджера, де втрата повідомлень неприпустима.
- Масштабованість: Підтримує різні стратегії масштабування, включаючи master-slave реплікацію, логічну реплікацію та горизонтальний шардинг.

2.2.7. Pillow

Pillow – це бібліотека Python для обробки зображень. У проекті вона використовується для завантаження та обробки аватарів, зокрема для створення мініатюр, зміни розмірів, перевірки цілісності зображень та конвертації у відповідні формати.

2.2.8. JavaScript WebSocket API

JavaScript WebSocket API – це нативний інструмент у браузері, який дозволяє встановлювати і підтримувати з'єднання з сервером у реальному часі. Він використовується для надсилання та отримання повідомлень у месенджері, забезпечуючи живу комунікацію між клієнтами.

3. Огляд застосунку

3.1. Ролі користувачів

У застосунку обміну повідомленнями передбачено дві основні ролі користувачів: звичайний користувач та адміністратор.

Звичайний користувач має можливість:

- Реєструватися в системі та проходити автентифікацію
- Приєднуватися до загального чату для обміну повідомленнями
- Надсилати та отримувати повідомлення в режимі реального часу
- Переглядати історію повідомлень
- Бачити статус активності інших користувачів (онлайн/офлайн)
- Редагувати свій профіль та завантажувати аватар

Адміністратор системи має розширені можливості:

- Управління користувачами через адміністративну панель Django
- Модерація повідомлень та контроль за дотриманням правил спілкування

- Моніторинг активності користувачів
- Налаштування системних параметрів
- Доступ до повної історії повідомлень та статистики використання
- Можливість блокування або видалення користувачів при порушеннях

3.2. Бізнес логіка застосунку

Логіка застосунку побудована навколо забезпечення швидкого та надійного обміну повідомленнями між користувачами в режимі реального часу через WebSocket-з'єднання.

3.2.1. Процес реєстрації та автентифікації

Новий користувач повинен пройти процедуру реєстрації, заповнивши форму з обов'язковими полями: ім'я користувача, електронна пошта, пароль та його підтвердження. Система автоматично перевіряє унікальність імені користувача та email адреси. Після успішної реєстрації створюється обліковий запис користувача і його профіль, тоді він може увійти до системи.

Процес автентифікації здійснюється через стандартну форму входу Django. Після входу користувач автоматично приєднується до загального чату і може почати обмін повідомленнями з іншими учасниками.

Реєстрація

Вже маєте обліковий запис? Тоді, будь ласка, [увійдіть](#).

Рис 5. Форма реєстрації застосунку

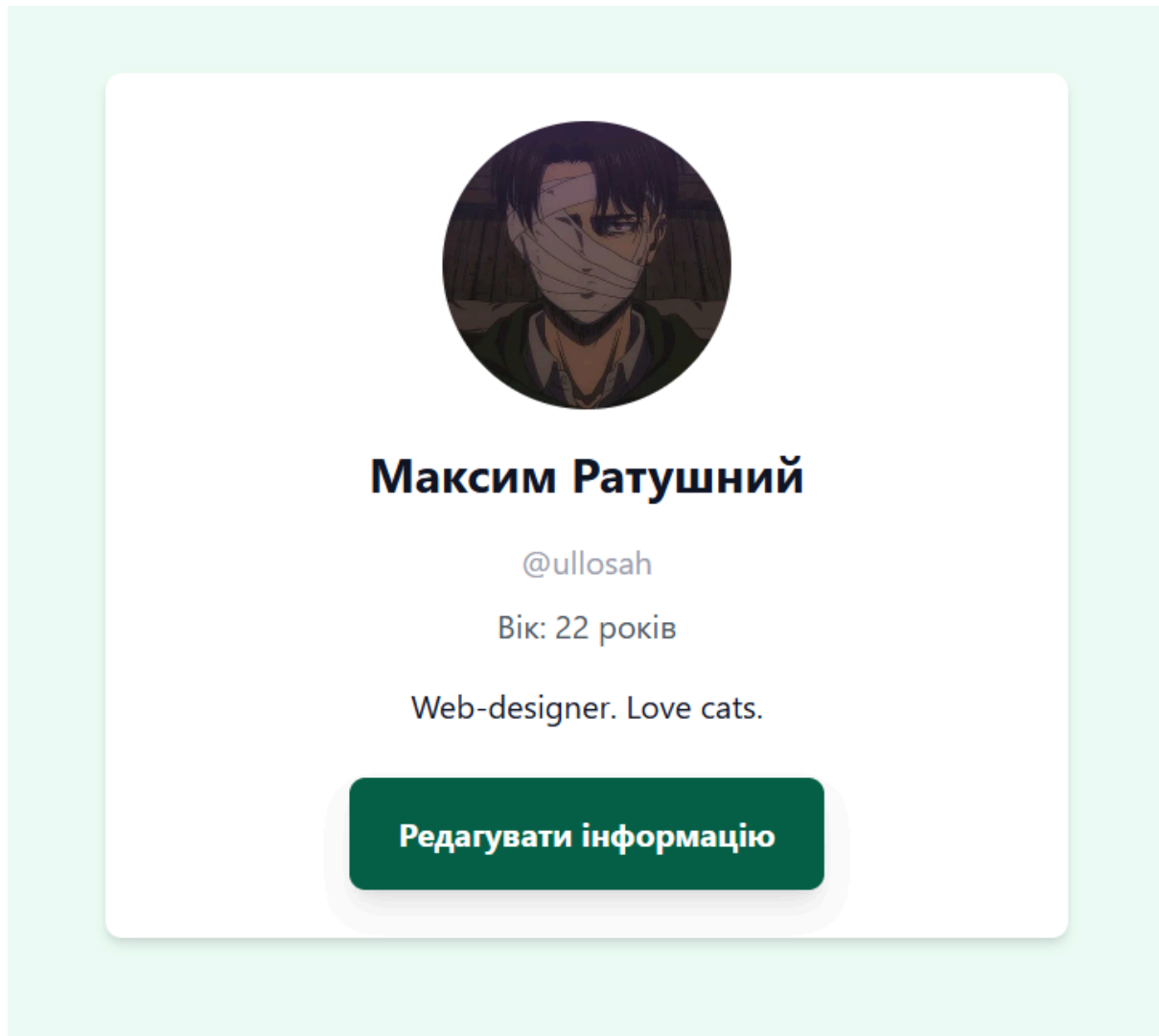


Рис 6. Профіль користувача

3.2.2. Обмін повідомленнями у реальному часі

Коли користувач надсилає повідомлення, воно передається через WebSocket-з'єднання до серверного Consumer'a, який обробляє повідомлення та зберігає його в базі даних PostgreSQL. Після збереження повідомлення негайно надсилається всім активним учасникам чату через Channel Groups.

Кожне повідомлення містить:

- Текст повідомлення
- Автора (посилання на модель User)

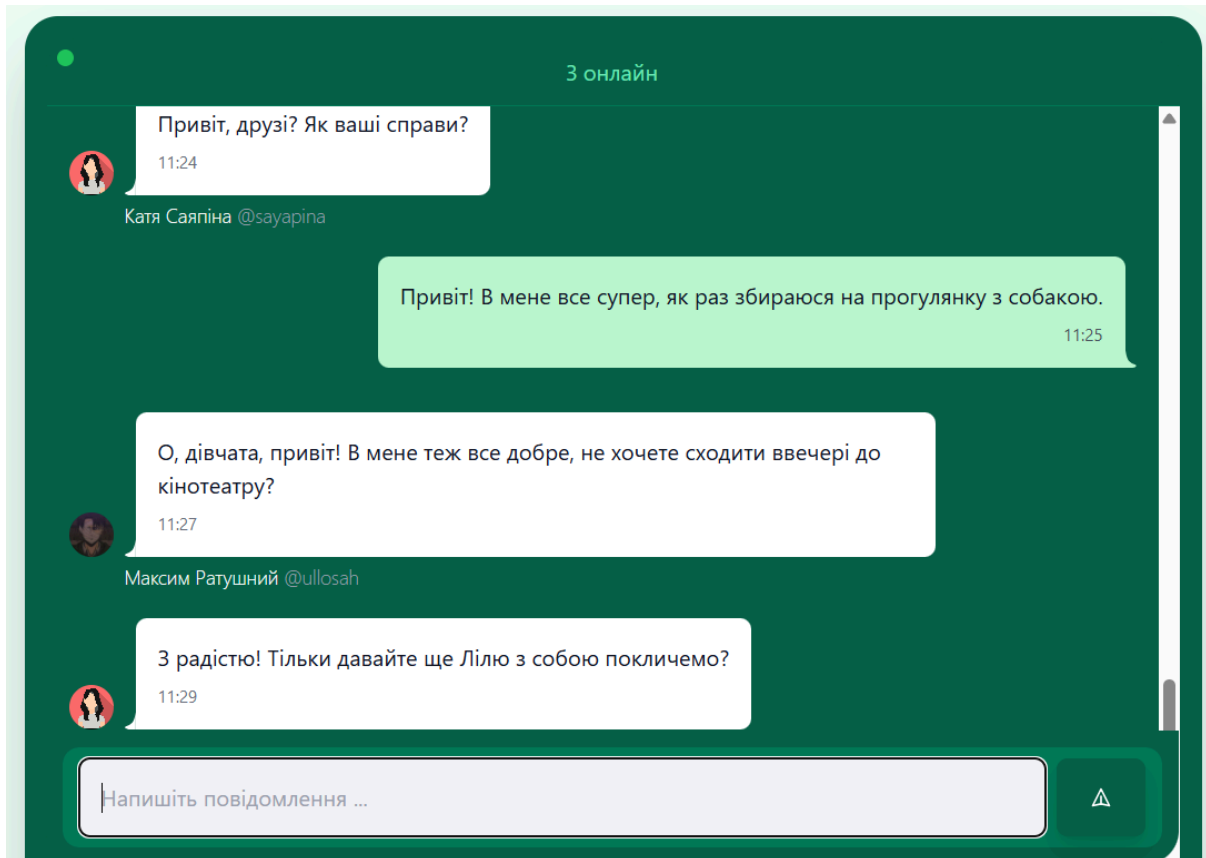


Рис 7. Груповий чат застосунку

3.2.3. Відображення статусу користувачів

Система відстежує статус активності користувачів через WebSocket-з'єднання:

- При підключенні користувача до чату його статус змінюється на "онлайн"
- При відключенні (закриття браузера, втрата з'єднання) статус змінюється на "офлайн"
- Статуси користувачів оновлюються у реальному часі для всіх учасників чату

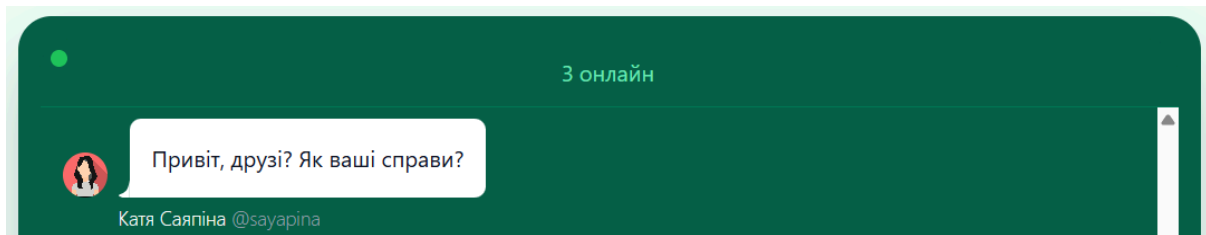


Рис 8. Статус активності користувачів

3.2.4. Історія повідомлень

Всі повідомлення зберігаються в базі даних і доступні для перегляду при повторному підключенні користувача. При завантаженні сторінки чату відображаються останні повідомлення, що дозволяє користувачам бачити контекст попередніх розмов.

3.3. Безпека та обмеження

3.3.1. Авторизація та доступ користувачів

Для забезпечення контролю доступу у застосунку реалізовано систему автентифікації на основі вбудованої моделі користувача Django. Кожен користувач повинен увійти до системи, використовуючи особисті облікові дані (email та пароль). Всі сторінки чату захищені декораторами `@login_required`, що означає доступність лише для авторизованих користувачів.

Система сесій Django забезпечує збереження стану автентифікації користувача між запитами. WebSocket-з'єднання також перевіряють автентифікацію користувача перед встановленням постійного з'єднання.

зареєструйтесь спочатку.' (If you don't have an account yet, please [register](#) first). There are two input fields: 'Адреса електронної пошти' (Email address) and 'Пароль' (Password). Below the password field is a link 'Забули пароль?' (Forgot password?). There is a checkbox labeled 'Запам'ятати Мене:' (Remember me:). At the bottom is a green button labeled 'Увійти' (Login)." data-bbox="118 85 879 437"/>

Увійти

Якщо у вас ще немає облікового запису, будь ласка, [зареєструйтесь](#) спочатку.

Адреса електронної пошти

Пароль

[Забули пароль?](#)

Запам'ятати Мене: ☐

Увійти

Рис 9. Форма входу застосунку

3.3.2. Рольові обмеження

У системі передбачено розмежування прав доступу відповідно до ролі користувача:

Звичайний користувач:

- Має доступ лише до інтерфейсу чату та власного профілю
- Може надсилати повідомлення та переглядати історію
- Не має доступу до адміністративних функцій
- Може редагувати лише власний профіль

Адміністратор:

- Має повний доступ до всіх функцій системи
- Може керувати користувачами через Django Admin
- Має можливість модерувати контент
- Може переглядати системну статистику та логи

3.3.3. Захист від загроз безпеки

Застосунок використовує вбудовані механізми захисту Django:

- CSRF-захист: Всі форми захищені від міжсайтових підробок запитів через використання CSRF токенів.
- XSS-захист: Вихідні дані автоматично екрануються в шаблонах Django для запобігання виконанню шкідливого JavaScript коду.
- SQL-ін'єкції: Використання Django ORM автоматично захищає від SQL-ін'єкцій через параметризовані запити.
- Валідація даних: Всі вхідні дані проходять валідацію як на клієнтській стороні (JavaScript), так і на серверній стороні (Django Forms).

3.3.4. Обмеження на розмір та тип контенту

Для забезпечення стабільної роботи системи встановлено наступні обмеження:

- Максимальна довжина повідомлення: 1000 символів
- Підтримувані формати аватарів: JPEG, PNG
- Максимальний розмір аватара: 5 МБ
- Автоматичне стиснення зображень до розміру 200x200 пікселів

Висновки

У ході виконання цієї курсової роботи було створено клієнт-серверний застосунок для обміну повідомленнями з використанням сучасних технологій, зокрема WebSocket, Django та PostgreSQL. У процесі розробки були прийняті важливі архітектурні рішення, які дозволили побудувати стабільну й ефективну систему для обміну повідомленнями в режимі реального часу.

Система забезпечує базову функціональність для взаємодії користувачів — реєстрацію, автентифікацію та обмін текстовими повідомленнями. У подальшому проєкт планується розширювати: зокрема, можна додати можливість обміну файлами, а також реалізувати можливість ведення особистих діалогів з користувачами. На даний момент результатом роботи є функціонуюча система групового чату з чітко розподіленою логікою між клієнтом і сервером, зручним інтерфейсом та стабільною взаємодією між компонентами.

Список використаних джерел:

1. Офіційна документація Django [Електронний ресурс]
<https://docs.djangoproject.com/en/5.2/> (Дата звернення:
17.05.2025)
2. Офіційна документація Django Channels [Електронний ресурс]
<https://channels.readthedocs.io/en/latest/> (Дата звернення:
17.05.2025)
3. Офіційна документація PostgreSQL [Електронний ресурс]
<https://www.postgresql.org/docs/> (Дата звернення: 18.05.2025)
4. Офіційна документація WebSocket Protocol [Електронний
ресурс]
<https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
(Дата звернення: 16.05.2025)
5. Офіційна документація HTMX [Електронний ресурс]
<https://htmx.org/docs/> (Дата звернення: 17.05.2025)

Додаток А

```

from channels.generic.websocket import WebsocketConsumer
from django.shortcuts import get_object_or_404
from django.template.loader import render_to_string
from asgiref.sync import async_to_sync
import json
from .models import *

class ChatroomConsumer(WebsocketConsumer):
    def connect(self):
        self.user = self.scope['user']
        self.chatroom_name = self.scope['url_route']['kwargs']['chatroom_name']
        self.chatroom = get_object_or_404(ChatGroup, group_name=self.chatroom_name)

        async_to_sync(self.channel_layer.group_add)(
            self.chatroom_name, self.channel_name
        )

        #add and update online users
        if self.user not in self.chatroom.users_online.all():
            self.chatroom.users_online.add(self.user)
            self.update_online_count()

        self.accept()

    def disconnect(self, close_code):
        async_to_sync(self.channel_layer.group_discard)(
            self.chatroom_name, self.channel_name
        )

        #remove and update online users
        if self.user in self.chatroom.users_online.all():
            self.chatroom.users_online.remove(self.user)
            self.update_online_count()

```

```

def receive(self, text_data):
    text_data_json = json.loads(text_data)
    body = text_data_json['body']

    message = GroupMessage.objects.create(
        body = body,
        author = self.user,
        group = self.chatroom
    )
    event = {
        'type': 'message_handler',
        'message_id': message.id,
    }
    async_to_sync(self.channel_layer.group_send)(
        self.chatroom_name, event
    )

def message_handler(self, event):
    message_id = event['message_id']
    message = GroupMessage.objects.get(id=message_id)
    context = {
        'message': message,
        'user': self.user,
    }
    html = render_to_string("a_rtchat/partials/chat_message_p.html", context=context)
    self.send(text_data=html)

```

```
def update_online_count(self):
    online_count = self.chatroom.users_online.count()

    event = {
        'type': 'online_count_handler',
        'online_count': online_count
    }
    async_to_sync(self.channel_layer.group_send)(self.chatroom_name, event)

def online_count_handler(self, event):
    online_count = event['online_count']
    html = render_to_string("a_rtchat/partials/online_count.html", {'online_count': online_count})
    self.send(text_data=html)
```

A.1 WebSocket Consumer для обробки повідомлень (consumers.py)

```
from django.urls import path
from .consumers import *

websocket_urlpatterns = [
    path("ws/chatroom/<chatroom_name>", ChatroomConsumer.as_asgi()),
]
```

A.2 Маршрутизація WebSocket (routing.py)

```

from django.db import models
from django.contrib.auth.models import User

class ChatGroup(models.Model):
    group_name = models.CharField(max_length=128, unique=True)
    users_online = models.ManyToManyField(User, related_name='online_in_groups', blank=True)

    def __str__(self):
        return self.group_name

class GroupMessage(models.Model):
    group = models.ForeignKey(ChatGroup, related_name='chat_messages', on_delete=models.CASCADE)
    author = models.ForeignKey(User, on_delete=models.CASCADE)
    body = models.CharField(max_length=300)
    created = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'{self.author.username} : {self.body}'

    class Meta:
        ordering = ['-created']

```

A.3 Модель даних для повідомлень (models.py)